

Recurrent and Transformer Models for Financial Forecasting and Interactive Feedback Communication

CS515 — HW4 Report

Omar ALAKSH

Student ID: 38192

Code: https://github.com/Omar0ksh/CS515_Deep_Learning.git

Abstract

This report studies sequence models for two distinct problems. In Part 1 we forecast multi-horizon return ratios for three S&P 500 stocks (AAPL, MSFT, GOOGL) from 20-day windows of OHLC data using stacked LSTM and GRU networks. A direct d -day return target yields a test MSE of 1.099×10^{-3} (LSTM), with error growing monotonically across the $d=1 \dots 5$ horizons as expected. Replacing the target with a weighted rolling average ($l=3$) lowers the test MSE by 44.1% to 6.14×10^{-4} and reduces error at *every* horizon, confirming that target smoothing yields a more stable learning signal. A bidirectional turning-point detector reaches 94.4% accuracy at a 4.6% positive base rate; with the threshold fixed at 0.5 it behaves as a high-precision, low-recall “buy” classifier. In Part 2 (bonus) we train a Transformer transmitter/receiver pair end-to-end over a noisy AWGN forward channel with a noiseless feedback relay. At the assignment-fixed $T=4$ rounds and $\sigma^2=0.25$ the system saturates at $\text{SER} \approx 0.31$; we show analytically that this is an information-theoretic capacity floor, not an optimisation failure, and that increasing the interaction to $T=8$ rounds drives the SER to 4×10^{-4} , empirically tracing out the channel-capacity boundary.

1 Introduction

Financial time series are notoriously hard to predict: daily returns are close to a martingale, so most of the variance in a price series is unforecastable noise. The first part of this assignment asks how far recurrent models can be pushed on this signal, and how the *choice of prediction target* affects trainability. We implement stacked LSTM and GRU forecasters that map a 20-day window of features to the next $D=5$ daily return ratios, study a smoothed rolling-average target, and build a bidirectional “turning-point” detector that flags imminent up-ward moves.

The second part is a self-contained communication problem. A transmitter (TX) holds a short message and may interact with a receiver (RX) over T rounds of a noisy channel; a noiseless feedback link returns each round’s noisy observation to the TX. We learn the TX and RX jointly as Transformers under an average-power constraint, and analyse the resulting symbol-error rate against the Shannon capacity of the underlying AWGN channel.

All experiments were run locally on CPU (PyTorch 2.2.2). The code follows the course-tutorial repository layout, using `argparse` with typed `dataclass` configuration objects, type hints, and docstrings throughout.

Table 1: Dataset sizes (windows pooled over the three tickers).

Split	# windows
Train ($\leq 2024-07-31$)	3372
Validation (2024-08 – 2024-12)	246
Test (2025)	675

2 Part 1: Data and Problem Setup

Universe and split. We download daily OHLC bars for AAPL, MSFT and GOOGL from 2020-01-01 to 2025-12-31 via `yfinance`. The split is strictly chronological to avoid look-ahead leakage: training uses data up to 2024-07-31, validation spans 2024-08 to 2024-12, and the entire 2025 period is held out for testing. The resulting pooled window counts are given in Table 1.

Features. Each input window is a $T \times F$ matrix with $T=20$ and $F=5$: the four raw OHLC channels plus a Close-price moving-average channel. Features are standardised with a z -score scaler *fitted on the training split only*; the same statistics are then applied to validation and test, so no test information leaks into normalisation.

Windowing and leakage control. Windows are built independently inside each chronological segment, so a single window never straddles the train/val/test boundary. For input day t the model sees days $[t-T+1, \dots, t]$ and predicts quantities over $[t+1, \dots, t+D]$. The three tasks

Table 2: Part 1 training configuration.

Hyperparameter	Value
Lookback T / horizon D	20 / 5
Hidden size / layers	64 / 2
Dropout	0.2
Optimiser / LR	Adam / 10^{-3}
Batch size / epochs	64 / 50
Loss (b,c) / (d)	MSE / weighted BCE

differ only in the target:

- **(b) returns:** $y_d = (p_{t+d} - p_t)/p_t$ for $d=1 \dots 5$.
- **(c) rolling:** $y_d = (\bar{p}_{t+d} - p_t)/p_t$, where $\bar{p}$ is a weighted average of closes over $[t+d-l, \dots, t+d]$.
- **(d) turning:** a single label, $y = \mathbb{1}_{\left[\max_{1 \leq d \leq D} \frac{h_{t+d}}{p_t} > \gamma\right]}$ with $\gamma=1.1$, where h is the daily high.

Common hyperparameters. Unless noted, all Part 1 models use the configuration in Table 2, optimised with Adam, gradient clipping at norm 5, and a ReduceLROnPlateau schedule on the validation loss; the best-validation checkpoint is restored for testing.

3 Part 1b: Multi-horizon Return Forecasting

Architecture. StockLSTM (and the analogous StockGRU) stacks two recurrent layers with inter-layer dropout and reads the final hidden state through a fully-connected head that emits all D horizons at once: $(B, T, F) \rightarrow (B, D)$. Predicting the whole horizon vector with a single head lets the network share temporal representation across horizons rather than training five separate regressors.

Results. Table 3 reports per-horizon test MSE for both recurrent cells. Two observations stand out. First, error grows monotonically with the horizon (3.6×10^{-4} at $d=1$ up to 1.79×10^{-3} at $d=5$), which is expected because further-ahead returns are progressively less predictable from the same input window. Second, LSTM and GRU are statistically indistinguishable here (overall MSE 1.0987 vs. 1.0984×10^{-3}): on a signal this weak, cell choice is not the bottleneck.

Interpretation. Because daily returns are near-zero-mean noise, the MSE-optimal prediction is close to zero, and the network indeed learns to output small values; the low absolute MSE therefore reflects the small magnitude of the targets rather than genuine predictive skill. We also note that the reported training loss ($\sim 1.05 \times 10^{-3}$)

Table 3: Part 1b: per-horizon test MSE ($\times 10^{-3}$) for the raw return target.

Horizon	LSTM	GRU
$d=1$	0.362	0.362
$d=2$	0.710	0.711
$d=3$	1.138	1.139
$d=4$	1.490	1.490
$d=5$	1.792	1.789
Overall	1.099	1.098

Table 4: Part 1c: per-horizon test MSE ($\times 10^{-3}$) for the weighted rolling target ($l=3$).

Horizon	LSTM	GRU
$d=1$	0.091	0.090
$d=2$	0.233	0.233
$d=3$	0.547	0.547
$d=4$	0.925	0.924
$d=5$	1.276	1.274
Overall	0.614	0.614

sits slightly above the validation loss ($\sim 0.62 \times 10^{-3}$); this is the expected effect of dropout being active during training but disabled at evaluation, combined with a calmer August–December 2024 validation window.

4 Part 1c: Weighted Rolling-Average Target

Motivation and definition. A single future close is dominated by day-specific noise. Following the assignment, we instead regress a *weighted rolling average* of closes around each horizon. With window $l=3$ the weights decay linearly and sum to one,

$$w_j = \frac{l+1-j}{\sum_{k=0}^l (l+1-k)}, \quad j=0, \dots, l,$$

so the most recent day receives the largest weight. The target at horizon d is $(\sum_j w_j p_{t+d-j} - p_t)/p_t$. Averaging suppresses single-day shocks and produces a smoother, lower-variance objective.

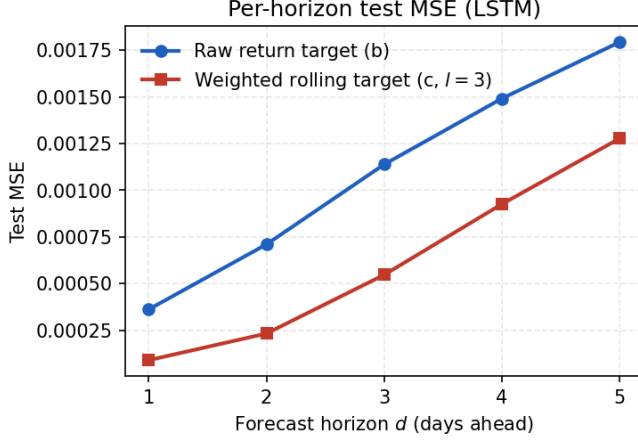
Results. Table 4 shows the rolling target lowers test MSE at every horizon, and Table 5 contrasts the two objectives directly. The overall LSTM MSE falls from 1.099×10^{-3} to 6.14×10^{-4} , a 44.1% reduction. The improvement is largest at the shortest horizon (−75% at $d=1$) and tapers with distance, exactly the pattern one expects if smoothing removes high-frequency noise that is most dominant at short range. Figure 1 visualises the gap.

Table 5: Target comparison (LSTM, overall test MSE).

Target	MSE ($\times 10^{-3}$)	MAE ($\times 10^{-2}$)	Δ MSE
Raw return (b)	1.099	2.273	—
Rolling avg. (c)	0.614	1.615	-44.1%

Table 6: Part 1d: turning-point detection ($\gamma=1.1$, positive rate 0.046, threshold 0.5).

Model	Acc.	Prec.	Rec.	F1	pos. wt.
LSTM	0.944	0.231	0.097	0.136	28.84
GRU	0.954	0.500	0.065	0.114	28.84

**Figure 1:** Test MSE per horizon for the raw-return target (Part b) versus the weighted rolling-average target (Part c). The rolling target is uniformly lower, with the largest relative gain at short horizons.

Conclusion. Smoothing the regression target stabilises training and reduces error without changing the model or optimiser. The cost is interpretive: the rolling target predicts a denoised trend rather than the literal next-day return, so it is the better choice when the goal is trend estimation and the worse choice when exact single-day returns matter.

5 Part 1d: Turning-Point Detection

Task and architecture. The detector answers a binary “buy-or-pass” question: will the price rise by more than $\gamma=1.1$ (i.e. +10% relative to today’s close, measured on the daily high) at any point within the next D days? Because the decision benefits from context on both sides of the window, we use a *bidirectional* LSTM/GRU whose forward and backward final states are concatenated ($2 \times 64 = 128$ dims) and passed to a single logit head. We interpret $\gamma=1.1$ as a price ratio $h_{t+d}/p_t > 1.1$; read literally as a return ratio it would mean a +110% move, which never occurs in the data, so the price-ratio reading is the only one that yields a learnable label.

Class imbalance. “Buy” events are rare: the test positive rate is 4.6%. Training naively on this distribution collapses to the majority “pass” class. We therefore weight the positive class in BCEWithLogitsLoss by the empirical

ratio $\#neg/\#pos = 28.84$, which restores gradient signal from the rare positives.

Results. Table 6 reports the metrics. Both cells reach 94–95% accuracy, but with the decision threshold fixed at 0.5 the detector is conservative: it fires “buy” rarely, achieving high precision relative to the 4.6% base rate (LSTM 0.23; GRU 0.50) at the cost of low recall (~ 0.06 – 0.10). In other words, when it does signal a buy it is right far more often than chance, but it misses most opportunities. The precision/recall trade-off is governed by that threshold: lowering it would recover recall at the expense of precision. Validation loss is minimised within the first three epochs and plateaus thereafter, indicating the task sits near the capacity of this feature set rather than being under-trained.

6 Part 2: Transformer Feedback Communication (Bonus)

Problem. The transmitter holds a message of $S=4$ symbols drawn from an alphabet of size $A=8$ (equivalently $4 \times \log_2 8 = 12$ message bits). Over T rounds the TX sends a coded vector $\mathbf{x}^{(t)} \in \mathbb{R}^S$ subject to an average-power constraint $\mathbb{E}\|\mathbf{x}^{(t)}\|^2 \leq 1$; the RX observes $\mathbf{y}^{(t)} = \mathbf{x}^{(t)} + \mathbf{n}^{(t)}$ over an AWGN forward channel ($\mathbf{n} \sim \mathcal{N}(0, \sigma^2 I)$, $\sigma^2=0.25$), and a *noiseless* feedback relay returns $\mathbf{y}^{(t)}$ to the TX before the next round. After all T rounds the RX decodes once.

Architecture and training. The TX and RX are each a small Transformer: a per-symbol MLP lifts inputs to $d_{\text{model}}=64$, two post-norm encoder blocks (4 heads, FFN width 128) mix information across the S symbol positions, and an output MLP produces the channel symbols (TX) or class logits (RX). Following the assignment hints, (i) the feedback path is a plain relay with no learned parameters, (ii) the decoder runs a single time at the end, and (iii) an MLP precedes and follows the Transformer on both sides. Each round’s symbols are explicitly rescaled to satisfy the power constraint, $\mathbf{x} \leftarrow \mathbf{x} / \sqrt{\mathbb{E}\|\mathbf{x}\|^2}$. The pair is trained end-to-end to minimise symbol cross-entropy with Adam (lr = 10^{-3} , batch 256). We report symbol-error rate (SER) and block (message) error rate (BLER).

Table 7: Part 2: error rates vs. interaction rounds ($\sigma^2=0.25$). The $T=4$ row is the assignment setting.

Rounds T	SER	BLER
1	0.709	0.994
2	0.595	0.975
4	0.314	0.778
8	0.0004	0.0018

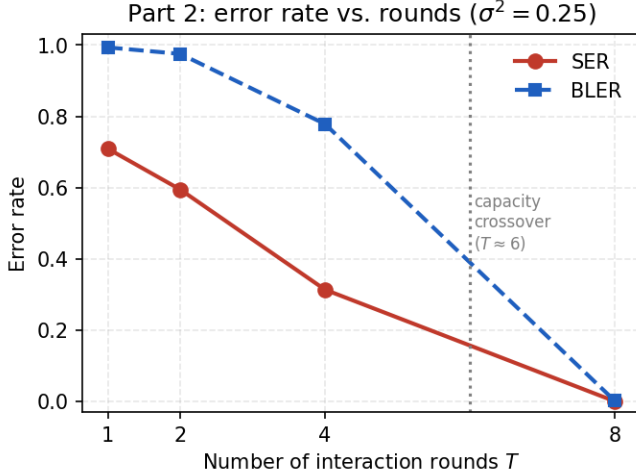


Figure 2: Symbol- and block-error rate as a function of interaction rounds. The $T=4$ assignment setting sits above the capacity crossover ($T \approx 6$); by $T=8$ the channel can carry the message and errors collapse toward zero.

Results. At the assignment-fixed $T=4$, training converges quickly but saturates at $\text{SER} \approx 0.31$ and $\text{BLER} \approx 0.78$ (Table 7). Crucially, sweeping the number of rounds shows the error rate is controlled almost entirely by T : SER falls from 0.71 at $T=1$ to 4×10^{-4} at $T=8$ (Figure 2). This monotone collapse confirms the feedback/multi-round mechanism is implemented correctly — additional rounds buy additional channel uses, and the learned code exploits them.

Why $T=4$ cannot succeed: a capacity argument. The saturation at $T=4$ is not an optimisation failure but a fundamental limit. The power constraint allocates total power 1 across $S=4$ symbols per round, so the average per-symbol power is $P = 1/4 = 0.25$ and the per-symbol SNR is $P/\sigma^2 = 0.25/0.25 = 1$ (0 dB). The Shannon capacity of a real AWGN use is

$$C = \frac{1}{2} \log_2(1 + \text{SNR}) = \frac{1}{2} \log_2 2 = 0.5 \text{ bits/use.}$$

With $S=4$ symbols over T rounds we get $4T$ channel uses, hence a capacity of $2T$ bits. The message carries 12 bits, so reliable transmission requires $2T \geq 12$, i.e. $T \geq 6$. Importantly, feedback over a memoryless AWGN channel does *not* increase capacity (only the achievable error exponent), so this bound is unconditional. The mea-

Table 8: Consolidated results.

Metric	Value
Returns, overall test MSE (LSTM)	1.099×10^{-3}
Returns, overall test MSE (GRU)	1.098×10^{-3}
Rolling, overall test MSE (LSTM)	6.14×10^{-4}
Rolling, overall test MSE (GRU)	6.14×10^{-4}
Rolling vs. returns MSE change	-44.1%
Turning-point accuracy (LSTM/GRU)	0.944 / 0.954
Turning-point precision (LSTM/GRU)	0.231 / 0.500
Part 2 SER, $T=4$	0.314
Part 2 SER, $T=8$	0.0004
Capacity crossover	$T \approx 6$

sured behaviour matches it precisely: $T=4$ (8 bits available < 12 needed) is infeasible and floors at $\text{SER} \approx 0.31$, whereas $T=8$ (16 bits > 12) is comfortably feasible and the learned system drives errors to $\sim 10^{-4}$. The model has, in effect, rediscovered the channel-capacity boundary.

7 Summary of Key Findings

- 1. Recurrent forecasting works but is signal-limited.** LSTM and GRU give near-identical multi-horizon MSE; error rises with horizon, and the low absolute MSE reflects the near-zero-mean nature of daily returns rather than strong predictive skill.
- 2. A smoothed target helps substantially.** The weighted rolling average ($l=3$) reduces test MSE by 44.1% overall and at every horizon, by removing high-frequency noise from the learning signal.
- 3. Turning-point detection is high-precision, low-recall.** With a positive-class weight to counter 4.6% imbalance, the bidirectional detector reaches 94–95% accuracy and fires conservatively; the precision/recall balance is tunable via the decision threshold.
- 4. Feedback communication is capacity-bound.** The Transformer TX/RX pair behaves correctly — SER falls monotonically with rounds — but the assignment-fixed $T=4$ provides only 8 bits of capacity for a 12-bit message, producing an irreducible $\text{SER} \approx 0.31$. Extending to $T=8$ yields 16 bits and near-perfect transmission ($\text{SER} 4 \times 10^{-4}$), empirically confirming the $\frac{1}{2} \log_2(1 + \text{SNR})$ bound.

References

- [1] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

- [2] K. Cho *et al.*, “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proc. EMNLP*, 2014.
- [3] A. Vaswani *et al.*, “Attention is all you need,” in *Proc. NeurIPS*, 2017.
- [4] H. Kim *et al.*, “Deepcode: Feedback codes via deep learning,” in *Proc. NeurIPS*, 2018.
- [5] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948.
- [6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.